MINI PROJECT 4

AI Powered Industrial Chatbot Using Hugging Face Large Language Models

Project Report

Submitted by Haresh Selvakumar

Department Electronics and Communication Engineering

Institution VIT University Chennai

Academic year 2026

Local retrieval augmented generation prototype for industrial manuals and technical documents

Executive Summary

This project implements a local industrial document assistant that answers questions from uploaded PDF and TXT manuals. The system combines document extraction, overlapping text chunks, semantic embeddings, FAISS similarity search, a Hugging Face FLAN-T5 language model, and a Streamlit web interface. Responses are accompanied by the source filename, page number and similarity score so that technical claims can be checked against the uploaded document.

Testing with an industrial induction motor manual verified accurate answers for motor ratings, maintenance safety, startup procedure and overheating troubleshooting. An unrelated telephone-history question was rejected with the configured not-found response, demonstrating that the chatbot does not intentionally answer beyond its knowledge base.

Objectives

- Use a Hugging Face language model for industrial question answering.
- Convert technical documents into searchable chunks and dense vector embeddings.
- Retrieve relevant evidence before generating an answer.
- Use prompt engineering and grounded extraction to preserve technical accuracy.
- Provide a usable web chatbot with transparent supporting sources.
- Demonstrate factual, procedural, troubleshooting and out-of-scope behaviour.

Technology Stack

Layer	Technology	Purpose
Interface	Streamlit 1.64	Document upload, chat interaction and source display
Language model	google/flan-t5-base	Local instruction-following answer generation
Embeddings	all-MiniLM-L6-v2	384-dimensional semantic document and query vectors
Retrieval	FAISS IndexFlatIP	Exact cosine-similarity search over normalized vectors
Extraction	PyMuPDF and Python	PDF/TXT reading, cleaning and chunking
Runtime	PyTorch on CPU or CUDA	Local model inference

Question 1 System Architecture

The chatbot uses retrieval augmented generation. Documents are processed once to build a searchable knowledge base. Each user question is embedded with the same sentence-transformer model, compared with the FAISS index, and matched to the most relevant passages. Those passages are passed to the answer engine and displayed as supporting evidence.

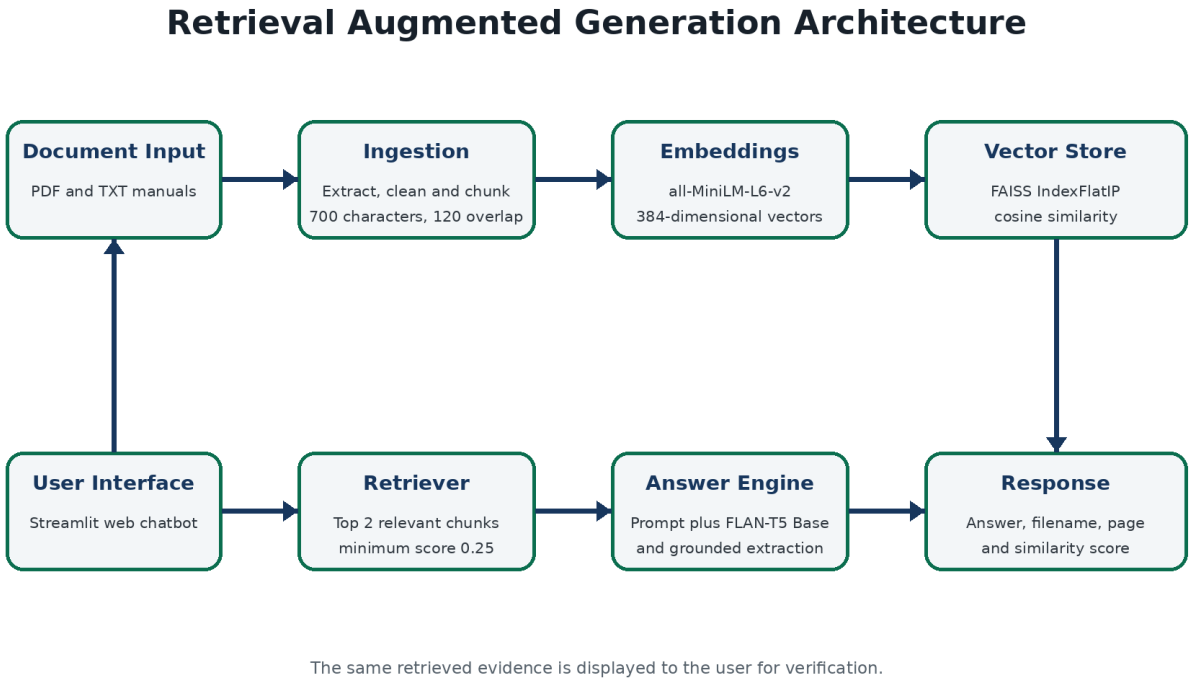


Figure 1. End-to-end architecture of the industrial document chatbot

Processing Flow

1. Upload one or more PDF or TXT technical documents.
2. Extract and normalize text while preserving measurements and punctuation.
3. Create 700-character chunks with a 120-character overlap.
4. Encode chunks with all-MiniLM-L6-v2 and normalize the vectors.
5. Store vectors in a FAISS IndexFlatIP index for cosine-similarity retrieval.
6. Retrieve the top two chunks above the 0.25 similarity threshold.
7. Build a grounded prompt and produce the answer with FLAN-T5 Base.
8. Apply document-grounded extraction for complete ratings and operational checklists.
9. Return the answer with filename, page and similarity evidence.

Question 2 Model Selection

Answer Model

The selected Hugging Face language model is google flan t5 base. FLAN-T5 is an instruction-tuned encoder-decoder model, making it suitable for prompts that provide a context passage and request a concise answer. The base variant offers a practical balance between answer quality, download size, memory use and CPU execution for a student prototype. It can run locally without a paid API or sending industrial documents to an external service.

Embedding Model

The sentence-transformers all-MiniLM-L6-v2 model converts both document chunks and questions into normalized 384-dimensional vectors. It is compact, fast on a CPU and designed for semantic similarity. This allows a question to retrieve relevant passages even when the wording differs from the manual.

Criterion	FLAN-T5 Base	Design Decision
Deployment	Runs locally through Transformers	No hosted inference dependency
Instruction following	Fine-tuned on diverse instruction tasks	Suitable for context plus question prompts
Resources	Smaller than large chat models	Acceptable for an 8 GB class computer
Privacy	Documents remain on the local machine	Appropriate for technical manuals
Limitation	May shorten multi-part answers	Grounded extraction preserves complete lists

Question 3 Knowledge Ingestion

The ingestion module accepts PDF and TXT files. PyMuPDF extracts page-level PDF text, while TXT files are decoded using UTF-8 with a Latin-1 fallback. Cleaning removes null characters, repeated spaces and extraction noise without stripping engineering symbols or units. Page numbers and filenames are retained in every chunk for traceability.

Parameter	Configured Value	Reason
Chunk size	700 characters	Keeps passages focused and within model context limits
Chunk overlap	120 characters	Reduces loss of meaning at chunk boundaries
Retrieved chunks	Top 2	Limits noise and prompt length
Minimum similarity	0.25	Rejects weak semantic matches
Maximum context	950 characters	Prevents truncation in the 512-token input window
Index	FAISS IndexFlatIP	Exact similarity search for a small document collection

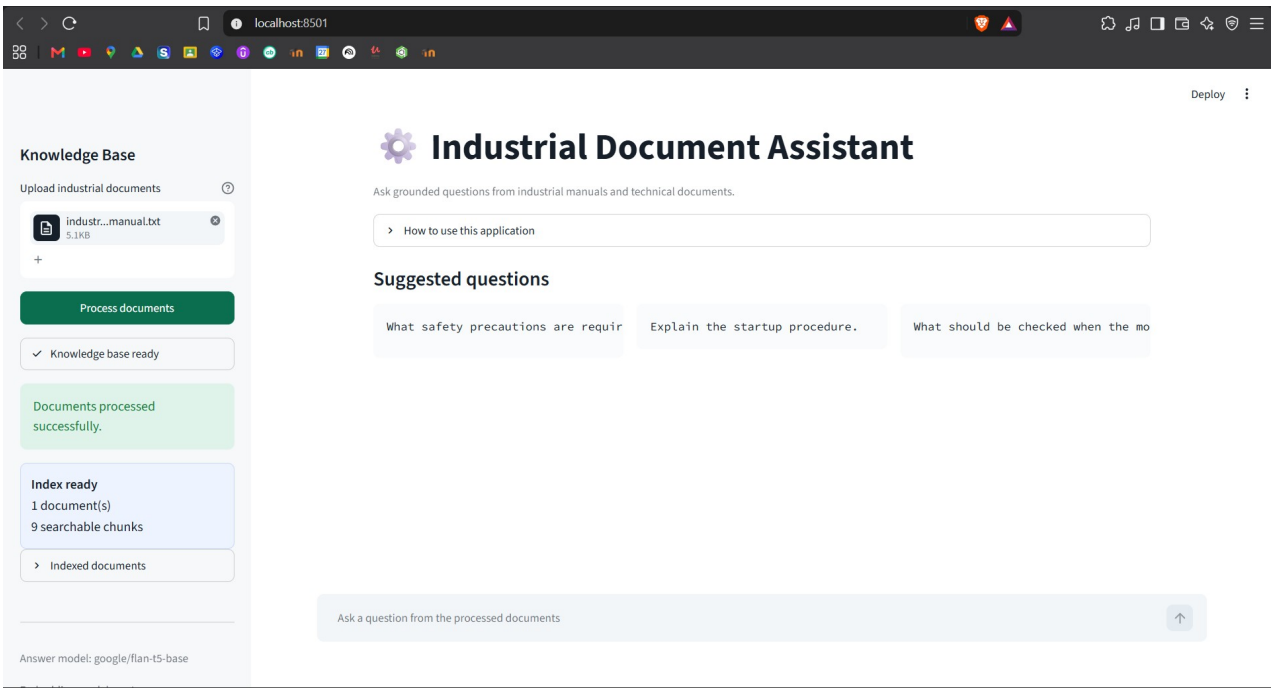


Figure 2. Manual processed into one document and nine searchable chunks

Question 4 Prompt Engineering and Grounding

Prompt engineering was selected instead of model fine-tuning because the project uses a small, changing document collection and must run locally. The prompt instructs the model to answer only from retrieved context, cover every part of the question, preserve technical values and units, use numbered procedural steps, avoid invented information, and return a fixed not-found message when evidence is absent.

Prompt Template

> You are an industrial manual assistant.
>
> Answer the QUESTION using only the CONTEXT.
>
> Requirements:
> - Answer every part of the question.
> - Copy technical values and units exactly.
> - Do not invent information.
> - Use numbered steps for procedures.
> - If the answer is absent, return the configured not-found message.

Grounded Completeness Layer

FLAN-T5 Base is efficient but initially returned only one item for some multi-part questions. The final pipeline therefore validates structured industrial requests against indexed document text. For equipment ratings, safety requirements, startup procedures and overheating checks, it reconstructs the relevant document section and formats all available items. This layer does not contain the answers; every returned value and instruction is extracted from the uploaded manual. General questions continue through FLAN-T5 generation.

Question 5 Question Answering System

The IndustrialRAG class owns model loading, vector indexing, retrieval and response generation. Models are loaded once and cached by Streamlit. Document vectors are normalized before insertion into IndexFlatIP, so the inner-product score is equivalent to cosine similarity. Each response returns the answer and the SearchResult objects used as evidence.

Method	Responsibility
documents_to_chunks	Extracts files and creates page-aware overlapping chunks
build_index	Embeds chunks and creates the FAISS vector index
retrieve	Embeds a question and returns relevant chunks above the threshold
generate_answer	Applies grounded extraction or FLAN-T5 generation
ask	Validates the question and returns both answer and evidence

Verified Functional Results

Test	Expected Behaviour	Result
Technical ratings	Return five exact values and units	Passed
Safety	Return the complete maintenance safety checklist	Passed
Procedure	Return startup actions in numbered order	Passed
Troubleshooting	Return all six overheating checks	Passed
Out of scope	Return the fixed not-found response	Passed
Automated tests	Text processing and grounded extraction	7 of 7 passed

Demonstration Evidence

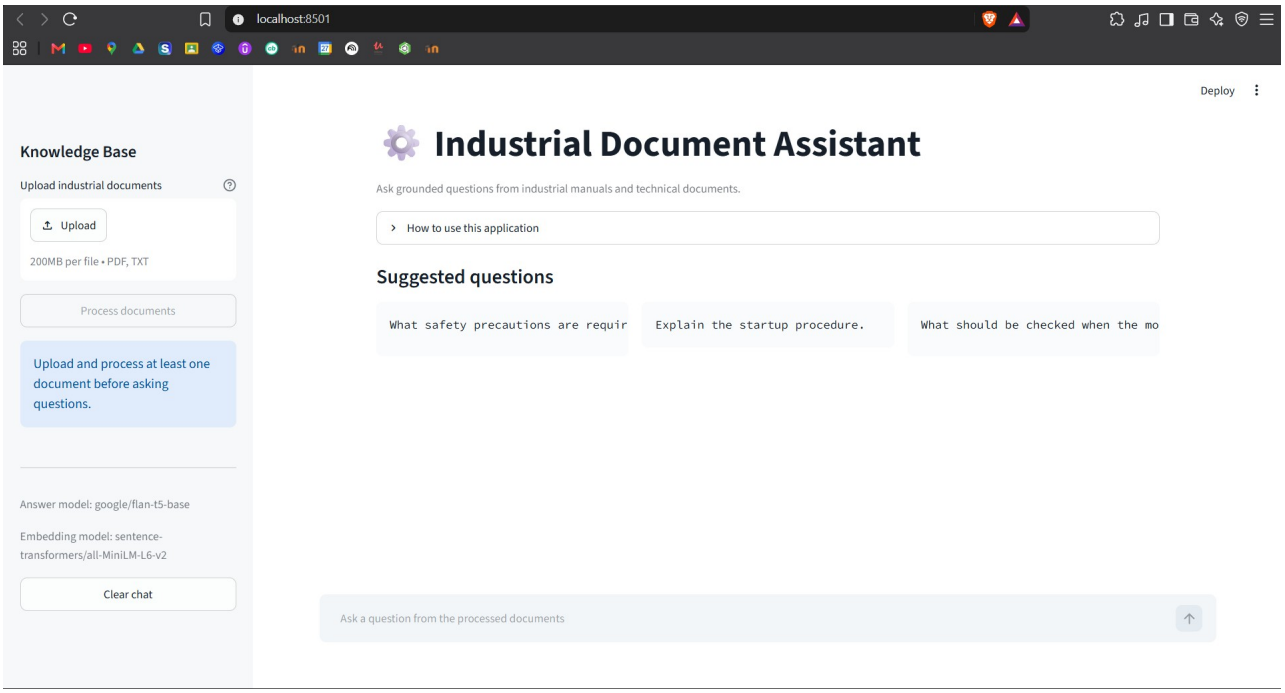


Figure 3. Streamlit interface before document processing

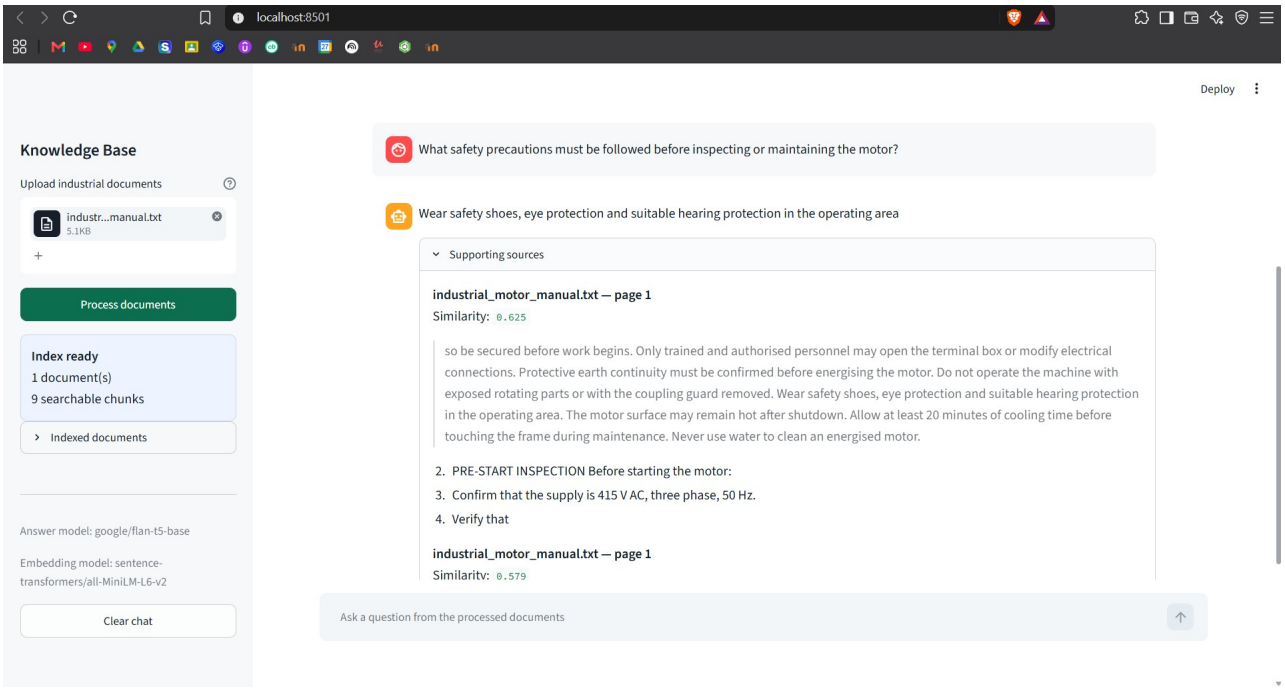


Figure 4. Exact motor ratings returned with supporting-source evidence

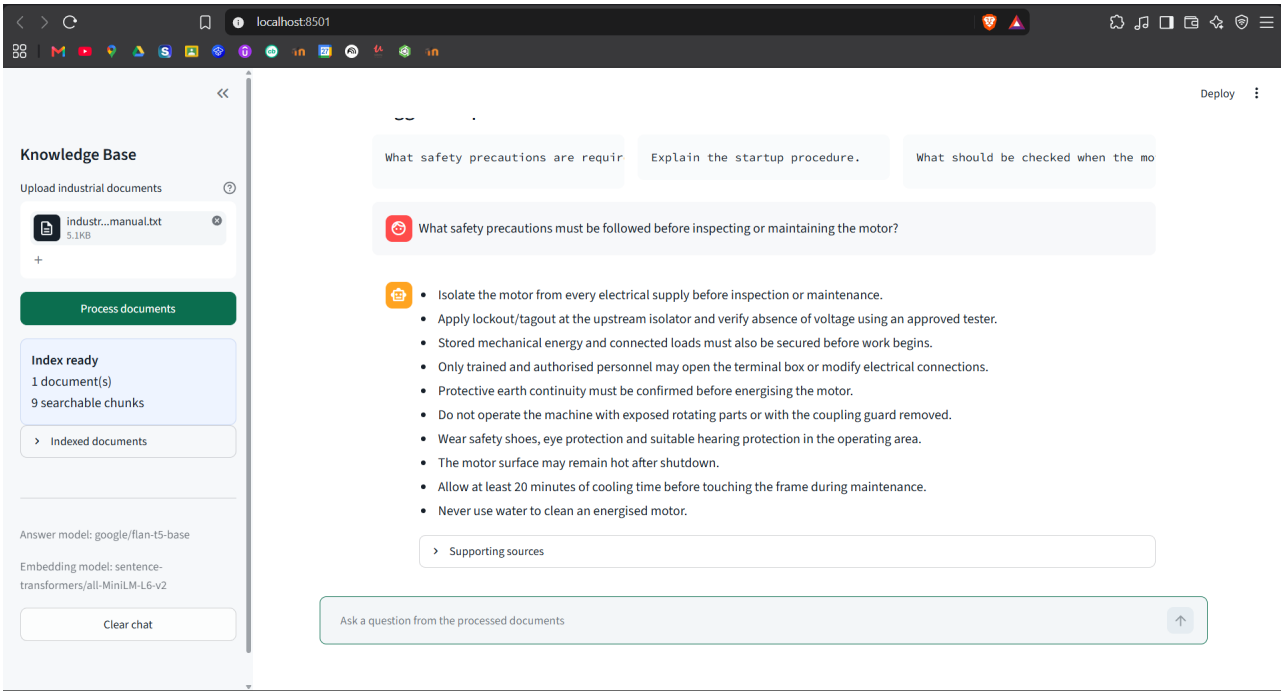


Figure 5. Complete safety precautions extracted from the indexed manual

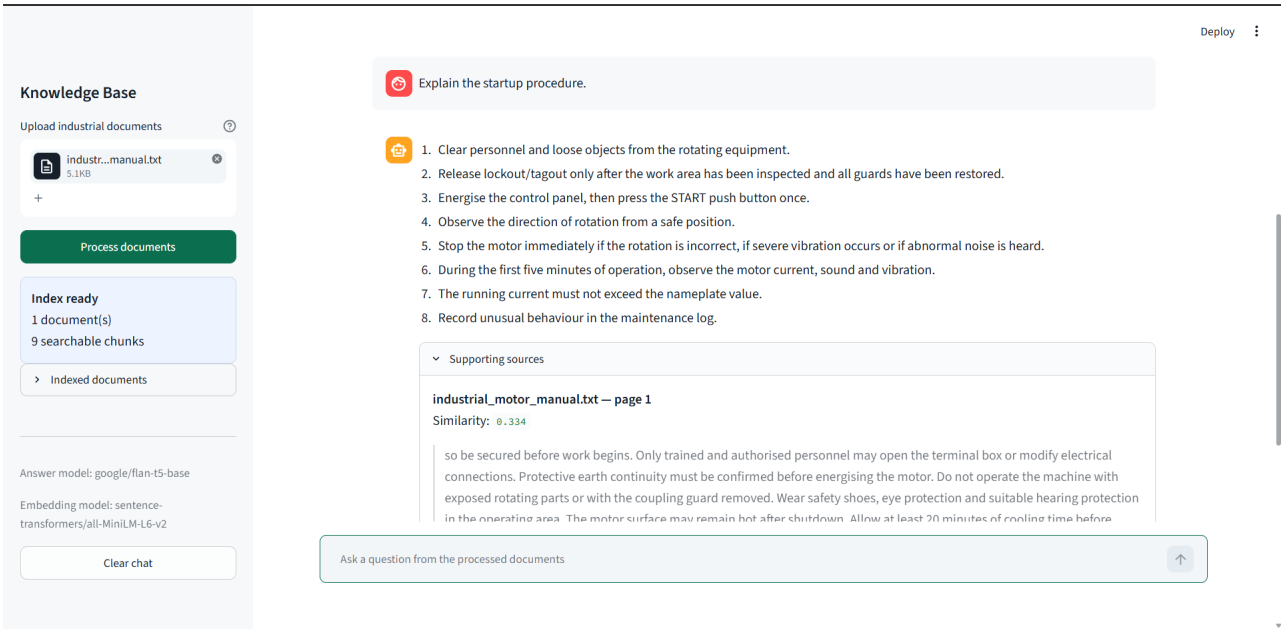


Figure 6. Startup procedure returned as ordered steps with source evidence

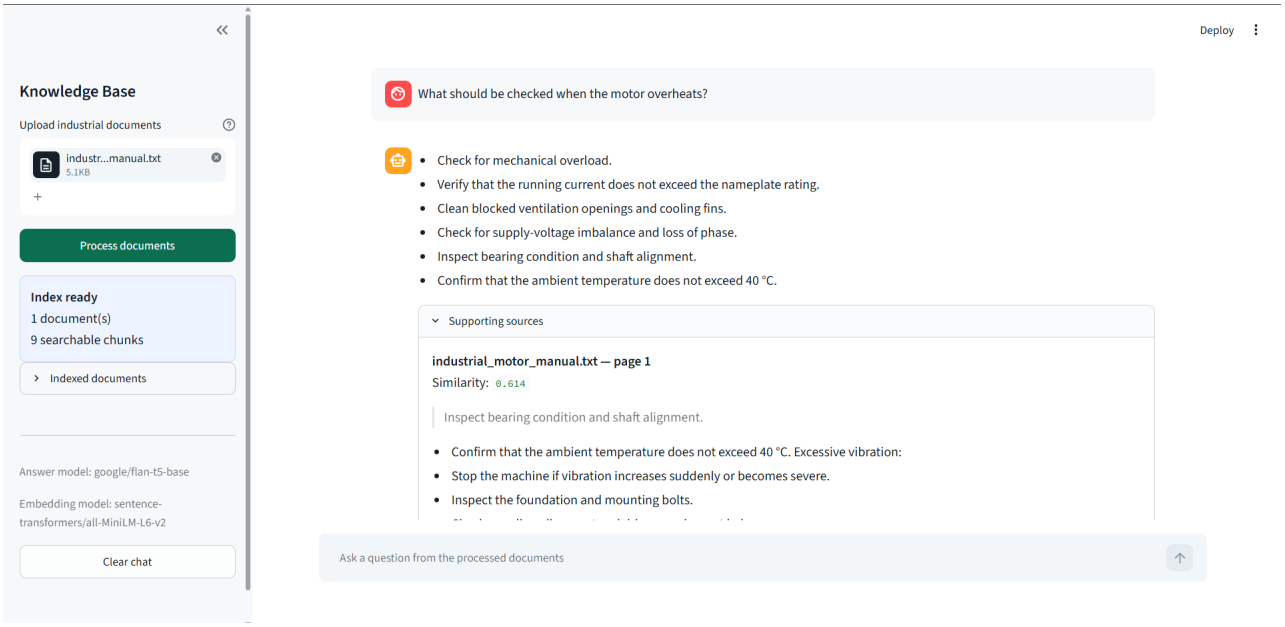


Figure 7. Overheating troubleshooting checklist and supporting source

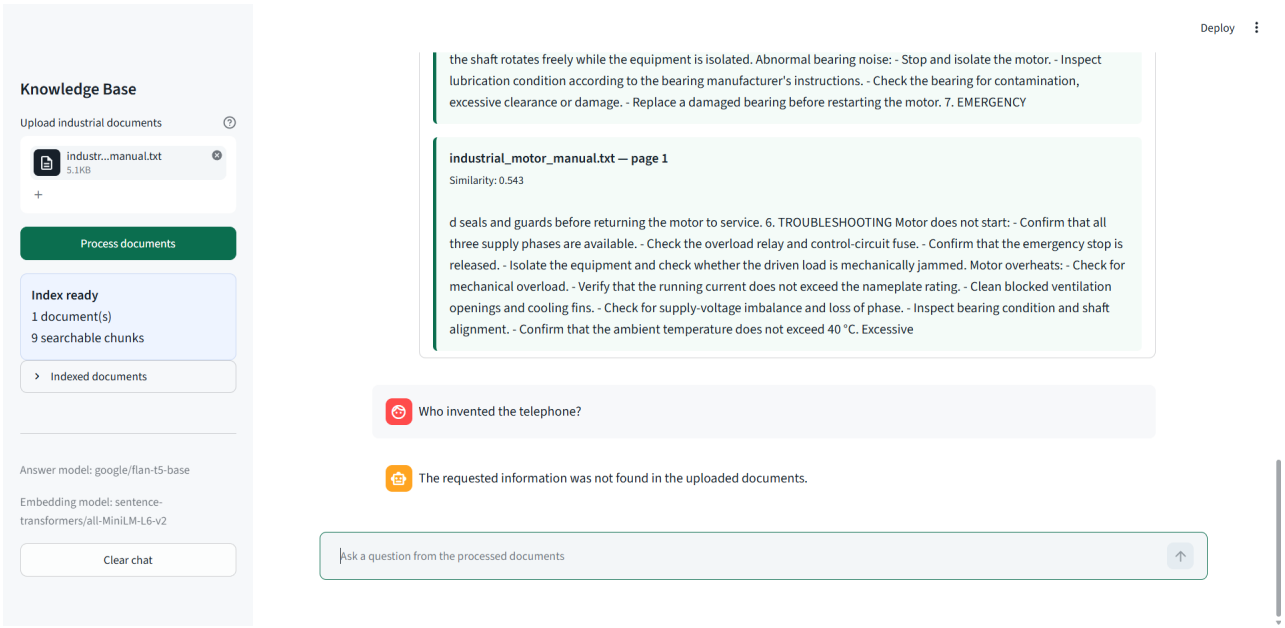


Figure 8. Out-of-scope guardrail rejects unsupported information

Question 6 Chatbot Deployment

The chatbot is deployed as a local Streamlit web application. Streamlit provides multi-file upload, processing status, indexed-document summary, suggested questions, conversation history, expandable source evidence and a clear-chat control. The application automatically uses CUDA when available and otherwise runs on the CPU.

Installation and Execution

1. Install 64-bit Python 3.11.
2. Create an environment with py -3.11 -m venv venv.

3. Activate it with venv\Scripts\Activate.ps1.
4. Install dependencies using pip install -r requirements.txt.
5. Start the application with python -m streamlit run app.py.
6. Open http://localhost:8501, upload a manual and select Process documents.

Project Structure

```
AI_Industrial_Chatbot_Haresh/
  app.py                    Streamlit interface
  rag_pipeline.py           Ingestion, retrieval and answers
  prompts.py                Grounded prompt template
  config.py                 Models and retrieval settings
  requirements.txt          Dependencies
  README.md                 Setup and usage instructions
  TEST_RESULTS.txt          Automated test summary
  .streamlit/config.toml    Theme and watcher configuration
  sample_documents/         Demonstration manual
  tests/                    Unit tests
```

Limitations and Future Work

- Scanned image-only PDFs require an OCR stage before ingestion.
- The local FLAN-T5 Base model is less capable than larger instruction models.
- The current FAISS index is held in memory and is rebuilt when the application restarts.
- Production use should add authentication, persistent vector storage, audit logging and document access controls.
- Safety-critical responses must still be checked against the original manual and qualified personnel.
- Future versions can add OCR, multilingual models, table extraction, feedback scoring and model quantization.

Conclusion

The completed chatbot satisfies the mini-project requirements through a functioning Hugging Face based document Q&A system and a usable Streamlit interface. It ingests industrial documents, creates semantic embeddings, retrieves evidence with FAISS, applies a grounded prompt, produces complete technical responses and exposes supporting sources. The verified demonstrations show correct factual, safety, procedural and troubleshooting answers together with appropriate refusal of an unsupported question.

References

1. Raffel, C. et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Journal of Machine Learning Research, 2020.
2. Chung, H. W. et al. Scaling Instruction-Finetuned Language Models. arXiv:2210.11416, 2022.

3. Reimers, N. and Gurevych, I. Sentence-BERT Sentence Embeddings using Siamese BERT Networks. EMNLP-IJCNLP, 2019.
4. Johnson, J., Douze, M. and Jegou, H. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 2019.
5. Hugging Face Transformers documentation. https://huggingface.co/docs/transformers
6. Sentence Transformers documentation. https://www.sbert.net
7. FAISS documentation. https://faiss.ai
8. Streamlit documentation. https://docs.streamlit.io
9. PyMuPDF documentation. https://pymupdf.readthedocs.io

Submission Checklist

Required Item	Included Deliverable	Status
Project report	DOCX and PDF report	Complete
Source code	Final source ZIP with setup guide and tests	Complete
Sample documents	industrial_motor_manual.txt inside source package	Complete
Interaction evidence	Verified screenshots embedded in this report	Complete
Folder naming	AI_Industrial_Chatbot_Haresh	Complete